

马尔可夫决策过程 (MDP)

先讲一个故事

想象你是一只小老鼠，住在一个 3×3 的迷宫里。你每次可以上、下、左、右走一步。走到奶酪格子得 +1 分，走到电击格子得 -1 分，其他格子不得分。你的目标是：**找到一条路，让长期累积的分数最多。**

这个故事就是一个 MDP。下面把故事里的要素翻译成数学语言。

五元组：形式化定义

MDP 由五个要素完整描述：

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$$

1. 状态空间 $\mathcal{S}$

所有可能状态的集合。在迷宫例子中， $\mathcal{S} = \{1, 2, \dots, 9\}$ (9 个格子)。

状态可以是离散的（如格子坐标）或连续的（如机器人关节角度）。 $|\mathcal{S}|$ 表示状态数量。

2. 动作空间 $\mathcal{A}$

所有可能动作的集合。 $\mathcal{A} = \{\uparrow, \downarrow, \leftarrow, \rightarrow\}$, $|\mathcal{A}| = 4$ 。

3. 状态转移概率 $\mathcal{P}$

这是 MDP 的 Markov 性质核心——下一状态 **只依赖当前状态和动作，与历史无关**：

$$P(S_{t+1} = s' \mid S_t = s, A_t = a, S_{t-1}, A_{t-1}, \dots, S_0) = P(S_{t+1} = s' \mid S_t = s, A_t = a)$$

简记为：

$$\mathcal{P}_{ss'}^a = P(s' \mid s, a)$$

对任意 s, a ，转移概率构成一个分布：

$$\sum_{s' \in \mathcal{S}} P(s' \mid s, a) = 1, \quad P(s' \mid s, a) \geq 0$$

老鼠例子：「往上走」可能 80% 真往上，10% 滑左边，10% 滑右边——这 **一个动作对应一个概率分布**，不是确定性的。

4. 奖励函数 $\mathcal{R}$

$R(s, a)$ 是在状态 s 采取动作 a 后获得的**即时奖励**：

$$R(s, a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$$

也可以是 $R(s, a, s')$ 形式（奖励同时依赖后继状态）。在迷宫例子中：踩奶酪 $R = +1$ ，踩电击 $R = -1$ ，其他 $R = 0$ 。

5. 折扣因子 $\gamma \in [0, 1)$

γ 控制「远见」程度：

- $\gamma \rightarrow 0$: 只看眼前奖励, 鼠目寸光;
- $\gamma \rightarrow 1$: 注重长期累积, 高瞻远瞩。

数学上, $\gamma < 1$ 保证了无限时域下回报级数收敛 (若奖励有界)。

交互过程的形式化描述

Agent 与环境在离散时间步 $t = 0, 1, 2, \dots$ 上交互。每一步产生一个四元组:

$$(S_t, A_t, R_{t+1}, S_{t+1})$$

完整轨迹 (trajectory):

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, \dots$$

这个序列的联合分布由转移概率 $\mathcal{P}$ 和策略 π 共同决定。这就是强化学习全部数据的来源——试错、拿反馈、调整、再来。

价值函数——什么是「好」状态？

回报 (Return)

你站在迷宫里, 想知道某个位置好不好。不能光看脚下的奖励——奶酪可能在隔壁, 也可能在十步之外。定义**折扣回报**:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

就是把未来的所有奖励加起来, 离现在越远折扣越重。

递归形式 (后面推导 Bellman 方程的关键):

$$G_t = R_{t+1} + \gamma G_{t+1}$$

这个恒等式是后续所有推导的起点。

价值函数: 期望的回报

因为转移是随机的, 你无法精确预测未来。定义**期望**:

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

$v_{\pi}(s)$ 是: 给定策略 π , 从状态 s 出发, **平均能拿多少回报**。

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$$

$q_{\pi}(s, a)$ 是: 在状态 s 先执行动作 a , 之后按照策略 π 行动, 平均能拿多少回报。

一句话: 价值不是精确的未来, 而是**对未来回报的条件期望**。

v_π 与 q_π 的关系

由定义立刻可得二者互推：

$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a | s) q_\pi(s, a)$$

(状态价值 = 按策略对动作价值加权平均)

$$q_\pi(s, a) = R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) v_\pi(s')$$

(动作价值 = 即时奖励 + 折扣后后继状态价值的期望)

这两个关系式在后面推导 Bellman 方程时会反复用到。

Bellman 期望方程——把未来拆成「眼前 + 之后」

核心直觉与推导

贝尔曼的关键发现是：**价值可以递归地定义**。从 $G_t = R_{t+1} + \gamma G_{t+1}$ 出发：

$$\begin{aligned} v_\pi(s) &= \mathbb{E}_\pi[G_t | S_t = s] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} | S_t = s] \\ &= \mathbb{E}_\pi[R_{t+1} | S_t = s] + \gamma \mathbb{E}_\pi[G_{t+1} | S_t = s] \end{aligned}$$

第一项是即时奖励的期望，第二项是后继价值的期望。对动作 a 和后继状态 s' 展开求和：

$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a | s) \left[R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) v_\pi(s') \right]$$

这就是 **Bellman 期望方程**（状态价值形式）。它说：**你不需要看到无限远的未来，只需看一步，然后用下一个状态的价值来估计余下的。**

同理可得动作价值的 Bellman 期望方程：

$$q_\pi(s, a) = R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) \sum_{a' \in \mathcal{A}} \pi(a' | s') q_\pi(s', a')$$

一个具体例子

假设你在迷宫格子 A，往右走必到格子 B，奖励 $R = 0$ 。如果 $v(B) = 5$ ， $\gamma = 0.9$ ：

$$q(A, \rightarrow) = 0 + 0.9 \times 5 = 4.5$$

格子 A 本身没奖励，但因为右边是「好地方」，所以 A 也是好地方。这就是 Bellman 方程的直观威力——**价值通过递归在状态间传播**。

矩阵形式（有限状态 MDP）

当状态空间有限时，Bellman 期望方程是**线性方程组**：

$$\mathbf{v}_\pi = \mathbf{R}^\pi + \gamma \mathbf{P}^\pi \mathbf{v}_\pi$$

其中 $\mathbf{v}_\pi \in \mathbb{R}^{|\mathcal{S}|}$, $R^\pi(s) = \sum_a \pi(a | s) R(s, a)$, $P^\pi(s, s') = \sum_a \pi(a | s) P(s' | s, a)$ 。

可直接求解：

$$\mathbf{v}_\pi = (\mathbf{I} - \gamma \mathbf{P}^\pi)^{-1} \mathbf{R}^\pi$$

这是唯一一个能写闭式解的地方——最优方程就没这么幸运了。

Bellman 最优方程——每一步都选最好的

最优价值函数

定义最优价值函数为所有策略中的**最大值**：

$$v_*(s) = \max_{\pi} v_{\pi}(s), \quad \forall s \in \mathcal{S} \quad q_*(s, a) = \max_{\pi} q_{\pi}(s, a), \quad \forall s \in \mathcal{S}, \forall a \in \mathcal{A}$$

最优策略 π_* 满足：对所有状态， $v_{\pi_*}(s) = v_*(s)$ 。这样的策略**一定存在**（对有限 MDP）。且：

$$v_*(s) = \max_a q_*(s, a)$$

状态价值的最优方程推导

如果老鼠训练了足够久，已经学会在每个状态选最好的动作，把 Bellman 期望方程中的「按策略加权平均」换成「取最大」：

$$\begin{aligned} v_*(s) &= \max_{a \in \mathcal{A}} q_*(s, a) \\ &= \max_a \left[R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) v_*(s') \right] \\ &= \max_{a \in \mathcal{A}} \left[R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) v_*(s') \right] \end{aligned}$$

这就是 **Bellman 最优方程**（状态价值形式）。和期望方程的区别只有一个： $\sum_a \pi(a | s)$ 换成了 $\max_a$ 。

更严格地从定义出发推导：

$$\begin{aligned} v_*(s) &= \max_{\pi} \mathbb{E}_{\pi} [R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s] \\ &= \max_a \left[R(s, a) + \gamma \sum_{s'} P(s' | s, a) \max_{\pi} v_{\pi}(s') \right] \\ &= \max_a \left[R(s, a) + \gamma \sum_{s'} P(s' | s, a) v_*(s') \right] \end{aligned}$$

动作价值的最优方程

同理：

$$q_*(s, a) = R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) \max_{a' \in \mathcal{A}} q_*(s', a')$$

为什么没有闭式解？

Bellman 期望方程——**线性**：

$$\mathbf{v}_\pi = \mathbf{R}^\pi + \gamma \mathbf{P}^\pi \mathbf{v}_\pi \implies \mathbf{v}_\pi = (\mathbf{I} - \gamma \mathbf{P}^\pi)^{-1} \mathbf{R}^\pi$$

Bellman 最优方程——**非线性**（含 max 算子）：

$$\mathbf{v}_* = \max_a [\mathbf{R}_a + \gamma \mathbf{P}_a \mathbf{v}_*]$$

max 破坏了线性结构，无法求逆。只能用迭代方法逼近（值迭代、策略迭代），这是后续章节的内容。

策略提升 (Policy Improvement)

一个重要的理论性质：给定任意策略 π 和它的 q_π ，构造新策略：

$$\pi'(s) = \arg \max_a q_\pi(s, a)$$

则 π' 不差于 π （即 $v_{\pi'} \geq v_\pi$ 逐分量），这称为**策略提升定理**。反复交替「评估 $\rightarrow$ 提升」至不动点，即收敛到最优策略 π_* 。

Bootstrapping —— 用猜测更新猜测

什么叫「自己举起自己」？

Bootstrapping 源于「pull oneself up by one's bootstraps」——用估计值更新估计值。

你第一次去一家餐厅：

- 点一道菜（试一个动作）
- 吃了一口觉得还行（得到即时奖励）
- 根据这一口的体验，**猜测**剩下的也会差不多（用 $V(s')$ 猜测 G_{t+1} ）
- 更新评价

你没把整盘菜吃完才下结论——而是**吃一口，猜余下，更新一次**。这就是 Bootstrapping。

数学定义

考虑价值函数更新的一般形式：

$$V(S_t) \leftarrow V(S_t) + \alpha \underbrace{[\text{Target}_t - V(S_t)]}_{\text{更新目标}}$$

若 Target_t 中包含**对后续状态的当前估值**，则是 bootstrapping；若 Target_t 完全由真实采样回报构成，则不是。

MC vs TD vs DP ——三种方法的数学对比

- MC (Monte Carlo) ——无 Bootstrapping

目标就是真实采样回报 G_t :

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)]$$

其中 $G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-t-1} R_T$ (到终止状态的完整和)。

必须等 episode 结束才能算 G_t ，无偏但方差高（一局输赢波动大）。

- TD(0) ——一步 Bootstrapping

目标用 $V(S_{t+1})$ 替代 G_{t+1} :

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

括号中的 $R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ 叫 **TD 误差 (TD error)**。 $V(S_{t+1})$ 是对 G_{t+1} 的有偏但低方差替代。**每步即可更新**，无需等 episode 结束。

- DP (Dynamic Programming) ——全 Bootstrapping

需要知道模型 $P(s' | s, a)$ ，对所有后继状态求期望：

$$V(s) \leftarrow \max_a \left[R(s, a) + \gamma \sum_{s'} P(s' | s, a) V(s') \right]$$

零方差（确定性计算），但**需要已知环境模型**。

TD(λ): 一步和多步之间

在 MC（等到底）和 TD(0)（只用一步）之间存在连续谱——TD(λ) 用**加权多步回报**：

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

其中 $G_t^{(n)}$ 是 n 步回报：

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$

$\lambda = 0$: 退化为 TD(0) (一步 bootstrapping)。 $\lambda = 1$: 退化为 MC (无 bootstrapping)。 $0 < \lambda < 1$: 在偏差和方差之间平滑调节。

偏差—方差权衡

	MC	TD(0)	DP
偏差	无偏	有偏	有偏（需模型）
方差	高	低	零
需模型	否	否	是
需等 episode 结束	是	否	—

Bootstrapping 的核心权衡：**引入有偏估计，换方差大幅降低和更快收敛**。TD 几乎总比 MC 学得快——这是 bootstrapping 在实践中的核心价值。

与 Bellman 方程的理论联系

Bellman 方程就是 bootstrapping 的数学根基：

$$v_{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s]$$

右端出现了 v_{π} 自身——**价值函数的定义包含了它自己**。TD 学习所做的，不过是将这个期望方程从「对所有后继求期望」改为「采样一次后继」来近似：

$$\underbrace{\mathbb{E}_{\pi}[R_{t+1} + \gamma V(S_{t+1})]}_{\text{Bellman 方程 (期望)}} \longrightarrow \underbrace{R_{t+1} + \gamma V(S_{t+1})}_{\text{TD 目标 (采样)}}$$

DP 用已知模型算期望；TD 用一次实际转移的采样替代模型。同一个递归结构，两种逼近方式。